# PhoneCam Technology Stack

PhoneCam is an Android-to-Windows virtual webcam system. It uses an Android mobile app to capture camera and microphone data, sends that data to a Windows PC, and exposes it as a normal webcam device for apps like Zoom, Microsoft Teams, OBS, browsers, and other video tools.

## High-Level Architecture

```
Android Phone
  CameraX + AudioRecord
        |
        v
  MediaCodec H.264 Encoder
        |
        v
  PhoneCam Binary Protocol
        |
        v
  USB ADB / WiFi TCP / Bluetooth RFCOMM
        |
        v
Windows PC
  C++ PhoneCamService
        |
        v
  FFmpeg Video Decoder + waveOut Audio Playback
        |
        v
  Win32 Shared Memory FrameBuffer
        |
        v
  DirectShow Virtual Camera Driver
        |
        v
  Zoom / Teams / OBS / Browser Apps
```

## Android Mobile App Technologies

| Technology | Used For | Functionality |
|---|---|---|
| Kotlin | Android app programming language | Main mobile app logic, services, camera handling, streaming, protocol handling, and UI state |
| Android SDK | Native Android platform APIs | Permissions, services, notifications, networking, Bluetooth, audio, power management, and app lifecycle |

| Technology | Used For | Functionality |
|---|---|---|
| Jetpack Compose | Mobile UI framework | Builds the Android app screen, connection controls, status text, and user controls |
| Material 3 | UI component design system | Provides modern Android UI styling and controls |
| CameraX | Camera capture library | Opens the phone camera, manages preview, handles front/back camera, and delivers frames for encoding |
| Camera2 API | Lower-level camera backend | Used internally through CameraX for camera hardware access |
| PreviewView | Camera preview rendering | Shows the live camera preview inside the Android app |
| ImageAnalysis | Frame pipeline | Supplies camera frames that are sent to the video encoder |
| MediaCodec | Hardware video encoder | Encodes raw camera frames into H.264 video for low-latency streaming |
| H.264 / AVC | Video compression format | Reduces video bandwidth before sending frames to the PC |
| AudioRecord | Microphone capture | Captures phone microphone audio as raw PCM samples |
| PCM 16-bit audio | Audio streaming format | Sends microphone audio to the PC for playback or microphone bridging |
| Foreground Service | Long-running Android service | Keeps streaming active while the app is running and shows a persistent notification |
| WakeLock | Power management | Helps keep the phone awake while streaming |
| Java/Kotlin Sockets | USB and WiFi transport | Runs a TCP server on the phone for PC connection |
| ServerSocket | TCP server | Listens for PC connections on port 4747 |
| Bluetooth RFCOMM | Bluetooth transport | Allows streaming without WiFi, using paired Bluetooth devices |
| Android Bluetooth APIs | Bluetooth server support | Starts the Bluetooth server and accepts PC Bluetooth connections |
| Kotlin Coroutines | Async/background work | Helps manage non-blocking work in the Android app |
| Gradle Kotlin DSL | Android build system | Builds debug/release APKs and app bundles |
| AndroidX Libraries | Android app support | Lifecycle, activity, Compose integration, compatibility helpers |

## Windows PC Technologies

| Technology | Used For | Functionality |
|---|---|---|
| C++17 | Native PC service and driver code | Handles transport, decoding, shared memory, audio playback, logging, and virtual camera internals |
| C# | Desktop control application | Implements the Windows PhoneCamUI app logic |
| WPF | Windows desktop UI framework | Builds the PC control panel for connection mode, quality, audio route, and camera controls |
| .NET 8 | PC UI runtime | Runs the WPF control application |
| Win32 API | Native Windows features | Shared memory, process control, audio APIs, sockets, handles, and system integration |
| Winsock2 | TCP networking | Connects the PC to the phone over USB-forwarded TCP or WiFi TCP |
| ADB | USB connection bridge | Forwards `tcp:4747` between PC and phone over USB |
| Android SDK Platform Tools | USB tooling | Provides `adb.exe` for device detection, APK install, permission grant, app launch, and port forwarding |
| Bluetooth RFCOMM | Bluetooth PC connection | Connects Windows to the phone over Bluetooth when WiFi/USB are not used |
| Windows Bluetooth APIs | Paired device discovery | Lists paired Bluetooth devices and connects to the selected phone |
| FFmpeg | Video decoding | Decodes H.264 video frames received from the phone |
| libavcodec | Codec decoding | Performs the actual H.264 decode work |
| libavformat | FFmpeg media support | Provides media-format utilities used by the PC service |
| libavutil | FFmpeg utility library | Provides common FFmpeg data structures and helpers |
| libswscale | Pixel format conversion | Converts decoded video frames into RGB/BGR format for the virtual camera |
| DirectShow | Virtual webcam framework | Makes PhoneCam appear as a normal Windows video capture device |
| COM DLL | Driver registration model | Registers the DirectShow virtual camera as a COM component |
| IBaseFilter / IPin | DirectShow interfaces | Implements the virtual camera filter and output pin |
| IAMStreamConfig | Camera format negotiation | Allows video apps to request supported resolutions and formats |

| Technology | Used For | Functionality |
|---|---|---|
| Win32 Named Shared Memory | Service-to-driver IPC | Transfers decoded frames from `PhoneCamService.exe` to `PhoneCamDriver.dll` |
| FrameBuffer | Shared frame storage | Stores latest RGB frame, width, height, FPS, and frame metadata |
| waveOut API | PC audio playback | Plays phone microphone audio through Windows output devices |
| WASAPI Loopback | Reverse audio capture | Captures PC speaker output and sends it back to the phone speaker mode |
| CMake | C++ build system | Builds `PhoneCamService.exe` and `PhoneCamDriver.dll` |
| Visual Studio / MSVC | Windows compiler toolchain | Compiles C++ service and DirectShow driver |
| Inno Setup | Installer generation | Creates Windows setup executables |
| PowerShell Scripts | Setup automation | Registers/unregisters driver, starts USB mode, and configures audio helper behavior |

## Shared Protocol Technologies

| Technology | Used For | Functionality |
|---|---|---|
| Custom binary protocol | Phone-to-PC communication | Defines how video, audio, control, heartbeat, and reverse-audio packets are exchanged |
| Big-endian byte order | Packet encoding | Keeps packet fields consistent across Android and Windows |
| Magic bytes `0xCAFE` | Packet validation | Helps identify valid PhoneCam packets |
| Packet header | Protocol structure | Stores packet type, flags, payload length, timestamp, and payload |
| TCP stream | USB/WiFi transport protocol | Carries ordered video/audio/control data between phone and PC |
| RFCOMM stream | Bluetooth transport protocol | Carries the same packet protocol over Bluetooth |
| Heartbeat packets | Connection health | Detects active/stalled connections |
| Control packets | Remote camera control | Allows PC to switch camera, rotate, autofocus, zoom, toggle flash, change resolution, and control audio |
| Timestamp fields | Media timing | Carries presentation time for video/audio frames |

# Main Runtime Functionalities

## Camera Streaming

The Android phone captures frames using CameraX, encodes them with MediaCodec into H.264, wraps them in PhoneCam video packets, and sends them to the PC.

On Windows, `PhoneCamService.exe` receives packets, decodes video using FFmpeg, converts frames into RGB/BGR, and writes them into shared memory. The DirectShow driver reads those frames and presents them as `PhoneCam Virtual Camera`.

## Audio Streaming

The phone captures microphone audio using AudioRecord. Audio is sent as PCM data packets to the PC. The PC service plays the audio using the Windows waveOut API.

The project also includes reverse-audio support where the PC can capture speaker output through WASAPI loopback and send it to the phone for playback.

## USB Connection

USB mode uses ADB port forwarding. The PC runs commands like:

```
adb forward tcp:4747 tcp:4747
```

After forwarding, the PC connects to `127.0.0.1:4747`, and ADB tunnels that TCP connection to the phone over USB.

## WiFi Connection

WiFi mode uses normal TCP sockets. The phone starts a server on port `4747`, displays its WiFi IP address, and the PC connects directly to that IP address.

## Bluetooth Connection

Bluetooth mode uses RFCOMM sockets. The phone starts a Bluetooth server with a matching UUID, and the PC connects to the paired phone using Windows Bluetooth APIs.

## Virtual Webcam

The Windows virtual webcam is implemented as a DirectShow source filter. Once registered, apps can select `PhoneCam Virtual Camera` the same way they select a physical webcam.

## PC Control Panel

The WPF app launches and controls `PhoneCamService.exe`. It sends text commands through the service process stdin. The service converts those commands into protocol control packets and sends them to the phone.

Examples of control commands:

```
flip
rotate
flash
focus-on
focus-off
keyframe
resolution 1280 720
bitrate 4000000
fps 30
audio-on
audio-off
```

## Build and Packaging Technologies

| Area | Technology | Output |
|---|---|---|
| Android build | Gradle + Android Gradle Plugin | APK and AAB |
| Android language build | Kotlin compiler | Android bytecode |
| PC service build | CMake + MSVC | `PhoneCamService.exe` |
| PC driver build | CMake + MSVC | `PhoneCamDriver.dll` |
| PC UI build | .NET SDK | `PhoneCamUI.exe` |
| Windows installer | Inno Setup | `PhoneCamSetup.exe` / `PhoneCamSetup-PC.exe` |
| Portable package | ZIP packaging | Portable Windows bundle |

## Important Project Files

| File/Folder | Purpose |
|---|---|
| `mobile-android/` | Android mobile app |
| `mobile-android/app/build.gradle.kts` | Android app dependencies and build configuration |
| `mobile-android/app/src/main/AndroidManifest.xml` | Android permissions, activity, and service declarations |
| `mobile-android/app/src/main/java/com/phonecam/camera/` | Camera, video encoder, audio capture, and phone speaker logic |
| `mobile-android/app/src/main/java/com/phonecam/transport/` | USB/WiFi TCP and Bluetooth transport servers |

| File/Folder | Purpose |
| --- | --- |
| `mobile-android/app/src/main/java/com/phonecam/protocol/` | Android-side protocol packet handling |
| `mobile-android/app/src/main/java/com/phonecam/service/` | Foreground streaming service |
| `mobile-android/app/src/main/java/com/phonecam/ui/` | Android Compose UI |
| `pc-client-windows/service/` | C++ PC service for transport, decoding, audio, and control |
| `pc-client-windows/driver/` | DirectShow virtual camera driver |
| `pc-client-windows/ui/PhoneCamUI/` | C# WPF desktop control app |
| `pc-client-windows/CMakeLists.txt` | C++ build configuration |
| `shared/protocol.md` | Protocol specification shared by Android and Windows |
| `tools/` | PowerShell helper scripts |
| `dist/` | Built installers, APKs, and portable packages |

## Summary

PhoneCam combines these main stacks:

- Kotlin and Android SDK for the mobile app.
- CameraX and MediaCodec for camera capture and H.264 encoding.
- AudioRecord for phone microphone capture.
- TCP sockets, ADB, and Bluetooth RFCOMM for device communication.
- C++17 and Win32 APIs for the Windows native service.
- FFmpeg for video decoding.
- DirectShow and COM for the virtual webcam driver.
- Win32 shared memory for fast service-to-driver frame transfer.
- C# WPF and .NET 8 for the Windows control UI.
- CMake, Gradle, MSVC, and Inno Setup for building and packaging.